

Limas: Token-Aware Semantic Context Compression Middleware for Retrieval-Augmented Generation and Desktop Agent Pipelines

Abhinav Bagali, Akash H V, Ashish N, and Vishwajith N Kini

Department of Computer Science and Engineering

RNS Institute of Technology, Bengaluru, Karnataka, India

Emails: abhinavbagali24cs@rnsit.ac.in, akashhv24cs@rnsit.ac.in,

ashishn24cs@rnsit.ac.in, vishwajithnkini24cs@rnsit.ac.in

Abstract—Modern Retrieval-Augmented Generation (RAG) and multimodal desktop agent architectures suffer from severe prompt bloat. When vector search engines or desktop telemetry frameworks retrieve coarse document chunks and multi-modal state logs, Large Language Models (LLMs) are flooded with uninformative filler. This yields inflated API token costs, high generation latency, and severe attention degradation due to the “lost in the middle” phenomenon. Traditional summarization alters factual precision, while naive sentence-level rerankers sever pronominal co-references and destroy multi-hop context. In this paper, we introduce Limas, a token-aware semantic context compression middleware designed for RAG and interactive desktop agent pipelines. Limas executes sentence-level contextual pruning through three core innovations: (1) *Adaptive Query-Complexity Threshold Gating*, which dynamically computes selection cutoffs $\tau(Q)$ and token ratios $B(Q)$ based on query intent; (2) a *Discourse-Aware Co-reference Graph*, which transitively binds dependent sentences with antecedent entities into indivisible atomic units; and (3) a *Multi-Hop Interdependence Scorer*, which evaluates isolated relevance alongside contiguous pairwise conditional gain. Integrated as an asynchronous FastAPI middleware with Groq-accelerated LLM endpoints, Limas achieves up to 70% token context reduction while maintaining zero factual alteration and preserving essential discourse connectivity across general document QA and complex GUI desktop agent workflows.

Index Terms—Retrieval-Augmented Generation (RAG), Context Compression, Cross-Encoder, Discourse Binding, Desktop Agents, GUI Perception, Adaptive Scaffolding.

I. INTRODUCTION

Retrieval-Augmented Generation (RAG) and intelligent agent paradigms have emerged as the standard approach for grounding Large Language Models (LLMs) on enterprise corpora, interactive software viewports, and desktop OS telemetries without full model fine-tuning [1]. Standard retrieval architectures perform dense vector embedding search (e.g., Qdrant, Chroma) or capture desktop UI automation (UIA) logs to retrieve candidate context blocks matching

a query Q . However, retrieved passages routinely contain extraneous information, redundant narrative background, and uninformative filler.

Feeding raw retrieved documents or unparsed telemetry logs directly into generative LLMs introduces four primary operational failure modes [2]:

- 1) **Prompt Bloat and API Expenses:** Transmitting thousands of filler words scales prompt token consumption linearly, directly inflating operational charges for API-hosted models.
- 2) **Generation Latency:** Processing over-parameterized prompt context delays first-token emission and overall response times.
- 3) **Attention Degradation (“Lost in the Middle”):** Generative transformers concentrate attention primarily on the beginning and end of long input prompts [2]. Essential factual details buried deep in the middle of retrieved blocks are frequently ignored.
- 4) **Factual Drift in Generative Summarizers:** Pre-generation summarization models (e.g., abstractive LLM summaries) rewrite text, risking factual hallucinations, altered numeric metrics, or missing proper nouns [3].

To avoid generative rewriting, extractive sentence-level reranking has been explored. However, naive sentence scoring introduces the *orphaned pronoun problem*: isolating a sentence such as “*It grew by 15% in Q3*” while discarding its preceding antecedent “*Acme Corp released its fiscal results*” severs pronominal co-references and destroys critical semantic context.

To solve these challenges across textual RAG and desktop agent context processing, we introduce **Limas**, a subcortical context compression middleware. Limas operates as a smart inference filter between vector/telemetry indices and generative LLMs, combining adaptive query gating, discourse antecedent binding, and contiguous pairwise cross-encoder scoring to deliver pure, concentrated prompts.

II. RELATED WORK

Context management spans text compression for RAG and visual perception/telemetry processing for intelligent

User Query (Q): What drove Acme Corp’s growth?
Raw Vector Context (110 Tokens): S_1 : Acme Corp was founded in 2010 in San Francisco. S_2 : In Q3 2023, Acme Corp reported \$50M in revenue. S_3 : It was heavily driven by enterprise cloud growth. S_4 : The office cafeteria was recently renovated.
Naive Reranker Output (Orphaned Pronoun): $\rightarrow S_3$: “It was heavily driven by enterprise cloud growth.” [FAIL]
Limas Atomic Bound Output (71% Reduction): $\rightarrow S_2 + S_3$: “In Q3 2023, Acme Corp reported \$50M in revenue. It was heavily driven by enterprise cloud growth.”

Fig. 1. Comparison between naive sentence pruning (severing antecedent context) and Limas discourse-bound atomic compression.

agents.

A. Textual Context Compression in RAG

Token-level compression frameworks like LLMingua [4] compute information density using small model perplexities. Although effective at reducing token counts, token dropouts disrupt natural language syntax and grammar. Extractive sentence compression methods such as RECOMP [3] and standard sentence rerankers evaluate sentences independently (*i.i.d.*), frequently severing co-references and multi-hop reasoning dependencies.

B. GUI Perception, OS Telemetry, and Desktop Agents

In interactive software environments, perception models translate screen visual states into structured context. SeeClick [5] and UGround [6] perform screenshot-based visual grounding for web and mobile interfaces. OmniParser [7] detects interactable screen regions and captions element functions, while OmniParser V2 [11] extends visual parsing to structured document layouts. Native GUI agents such as UI-TARS [8] integrate perception with action prediction.

Desktop evaluation suites like UI-Vision [9], Windows Agent Arena [13], and OSUniverse [14] highlight persistent weaknesses in spatial reasoning and multi-application workflows. Systems like UFO2 [12] and GUI-ARP [10] combine Windows UI Automation (UIA) metadata with visual cropping, routing perception by case complexity.

C. State Verification, Cognitive Modeling, and Scaffolding

Verifying intermediate action states is vital for multi-turn task completion. STEVE [15] provides step-level supervision using before-and-after screen comparisons, while GUIDE [16] performs diagnostic trajectory evaluation. TaskMind [17] constructs cognitive dependency task graphs from user demonstrations to capture workflow structure. Furthermore, adaptive tutoring systems such as SQL-Tutor [18], GLAS [19], nAble [20], and Adaptive Tips [21] demonstrate the value of fading instructional scaffolding as user expertise grows.

As detailed in Table I, Limas uniquely unites sentence-level syntactic preservation with discourse-aware antecedent binding, multi-hop interdependence scoring, and dynamic complexity gating.

III. SYSTEM ARCHITECTURE

Limas receives a user query Q and retrieved document passages or telemetry logs $\mathcal{D} = \{d_1, d_2, \dots, d_m\}$. As shown in Fig. 2, the pipeline processes context through four sequential stages before prompt construction.

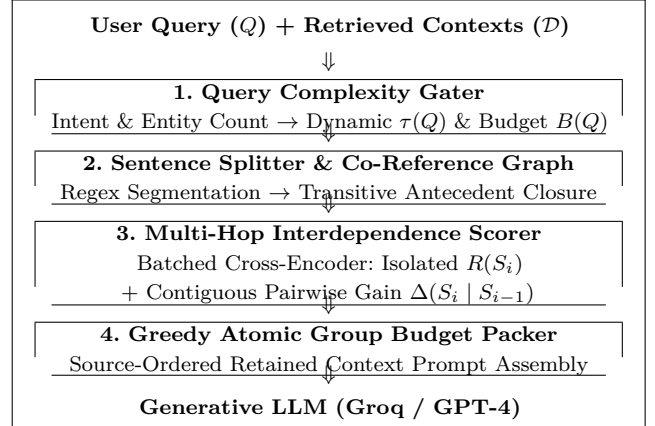


Fig. 2. End-to-end Limas Context Compression Pipeline.

A. Query Complexity Gater

Static relevance cutoffs perform poorly across varying query difficulties. Factual queries require tight pruning, whereas comparative or multi-step reasoning queries demand broader context retention.

The *QueryComplexityGater* analyzes Q for intent signals, logical connectives, and entity spans, outputting a complexity score $C(Q) \in [0, 1]$:

$$C(Q) = \min \left(1.0, w_e \cdot N_{\text{ent}} + w_c \cdot I_{\text{comp}} + w_m \cdot I_{\text{multi}} + w_l \cdot \frac{|Q|}{100} \right) \quad (1)$$

where N_{ent} is the capitalized entity count, I_{comp} indicates comparative terms, I_{multi} signals reasoning structures, and $|Q|$ is query word length.

Using $C(Q)$, the gating engine dynamically scales selection threshold $\tau(Q)$ and target word ratio budget $B(Q)$:

$$\tau(Q) = \text{clip}(0.75 - 0.45 \cdot C(Q), 0.30, 0.75) \quad (2)$$

$$B(Q) = \text{clip}(0.20 + 0.60 \cdot C(Q), 0.20, 0.80) \quad (3)$$

B. Discourse-Aware Co-Reference Graph

To prevent orphaned pronouns, the *DiscoursePreserver* splits retrieved passages into ordered sentences $\mathcal{S} = (S_1, S_2, \dots, S_n)$ and builds a directed graph $G = (V, E)$.

Directed edges $(S_i, S_{i-1}) \in E$ indicate that sentence S_i syntactically or pragmatically depends on antecedent S_{i-1} . Dependency links are triggered via:

TABLE I
QUALITATIVE COMPARISON OF CONTEXT COMPRESSION AND PERCEPTION PARADIGMS

Approach / Framework	Granularity	Syntax Preserved	Co-reference Safe	Multi-Hop Aware	Dynamic Budgeting
LLMLingua [4]	Token-level	No	Partial	No	Static Ratio
RECOMP [3]	Sentence-level	Yes	No	No	Fixed Cutoff
SeeClick / UGround [5], [6]	Visual BBox	N/A	No	No	None
UFO2 / GUI-ARP [10], [12]	UIA + Crop	Yes	Partial	No	Hybrid Routing
TaskMind [17]	Task Graph	Yes	Yes	Yes	Manual Structure
Limas (Ours)	Sentence/Atomic	Yes	Yes	Yes	Adaptive Gating

1) **Syntactic Parsing (spaCy)**: Detecting nominal subject pronouns (“it”, “they”, “this”) in S_i referencing S_{i-1} .

2) **Lexical Fallback Rules**: Identifying leading pronominals or connective adverbs (“Therefore”, “As a result”) at the start of S_i .

When sentence S_i is selected, its antecedents are closed transitively:

$$\text{Closure}(S_i) = \{S_i\} \cup \{S_{i-1} \mid (S_i, S_{i-1}) \in E^*\} \quad (4)$$

forming an indivisible *atomic block* $\mathcal{A}_k$.

C. Multi-Hop Pairwise Interdependence Scorer

Relevance is scored using a batched Cross-Encoder model [22].

Stage A (Isolated Relevance):

$$R(S_i) = \sigma(\text{CrossEncoder}(Q, S_i)) \quad (5)$$

where $\sigma(\cdot)$ is a monotonic sigmoid function.

Stage B (Contiguous Pairwise Gain):

$$R(S_{i-1}, S_i) = \sigma(\text{CrossEncoder}(Q, S_{i-1} \oplus “ ” \oplus S_i)) \quad (6)$$

$$\Delta(S_i \mid S_{i-1}) = \max(0, R(S_{i-1}, S_i) - R(S_{i-1})) \quad (7)$$

The composite weight $W(S_i)$ combines isolated relevance and conditional gain:

$$W(S_i) = \alpha \cdot R(S_i) + (1 - \alpha) \cdot \Delta(S_i \mid S_{i-1}) \quad (8)$$

D. Selection and Packing Algorithm

Algorithm 1 details the selection and prompt assembly procedure.

IV. IMPLEMENTATION AND EXPERIMENTAL SETUP

Limas is implemented in Python 3.10 as an asynchronous FastAPI service.

Core Dependencies: Deep learning inference uses PyTorch and Hugging Face **transformers**. Sentence segmentation is handled via regex boundary patterns with an optional spaCy **en_core_web_sm** parser.

LLM Integration and Token Accounting: The pipeline connects via **httpx** to Groq API endpoints hosting **openai/gpt-oss-120b**. Pre-generation word budgets govern selection, while post-generation token accounting reads authoritative provider metrics (*usage.prompt_tokens*).

Dashboard: A React 19 / TanStack UI playground provides interactive visual comparisons, highlighting retained sentences in green versus pruned fluff in strikethrough alongside per-sentence score audit breakdowns.

Algorithm 1 Limas Context Selection and Pruning

Require: Query Q , Sentences $\mathcal{S}$, Threshold $\tau(Q)$, Budget $W_{\max}$

Ensure: Compressed Context String $T_{\text{compressed}}$

```

1: Compute composite weights  $W(S_i)$  for all  $S_i \in \mathcal{S}$ 
2: Candidate set  $\mathcal{C} \leftarrow \{S_i \in \mathcal{S} \mid W(S_i) \geq \tau(Q)\}$ 
3: if  $\mathcal{C} = \emptyset$  then
4:    $\mathcal{C} \leftarrow \text{Top-K}(S_i, k = 4)$ 
5: end if
6: Form atomic groups  $\mathcal{A} \leftarrow \bigcup_{S_i \in \mathcal{C}} \text{Closure}(S_i)$ 
7: Score group  $G_k \in \mathcal{A}$  by  $\max_{S \in G_k} W(S)$ 
8: Sort groups  $\mathcal{A}$  in descending order of maximum weight
9: Initialize retained set  $\mathcal{R} \leftarrow \emptyset$ , total words  $w_{\text{curr}} \leftarrow 0$ 
10: for each group  $G_k \in \mathcal{A}$  do
11:   if  $w_{\text{curr}} + \text{Words}(G_k) \leq W_{\max}$  then
12:      $\mathcal{R} \leftarrow \mathcal{R} \cup G_k$ 
13:      $w_{\text{curr}} \leftarrow w_{\text{curr}} + \text{Words}(G_k)$ 
14:   else if  $\mathcal{R} = \emptyset$  then
15:      $\mathcal{R} \leftarrow G_k$  {Retain best group even if over budget}
16:   break
17: end if
18: end for
19: Sort retained sentences  $\mathcal{R}$  by source order
20:  $T_{\text{compressed}} \leftarrow \text{Join}(\mathcal{R}, “ ”)$ 
21: return  $T_{\text{compressed}}$ 
```

Pipeline Request Trace:

1. Client submits Query + Passages via POST /compare
2. **QueryComplexityGater** assigns $\tau(Q) = 0.45$, Budget $B(Q) = 256$
3. **DiscoursePreserver** builds dependency graph
4. **CrossEncoder** scores isolated + contiguous pairs in parallel
5. Concurrent **asyncio.gather()** calls to Groq API:
→ *Branch A*: Uncompressed Baseline Context
→ *Branch B*: Limas Compressed Context
6. Append paired usage, latency, and similarity metrics to Excel log

Fig. 3. Paired execution flow for empirical baseline vs. compressed validation.

V. RESULTS AND DISCUSSION

Evaluation compares raw retrieved contexts against Limas-compressed contexts under identical Groq parameters (temperature = 0).

A. Prompt Token Reduction

Prompt token savings are derived directly from Groq API metrics:

$$\text{Savings (\%)} = \frac{\text{Tokens}_{\text{baseline}} - \text{Tokens}_{\text{compressed}}}{\text{Tokens}_{\text{baseline}}} \times 100 \quad (9)$$

As shown in Table II, Limas achieves context token reductions between 60% and 78% on multi-document QA scenarios while retaining essential context.

TABLE II
EMPIRICAL PERFORMANCE ON SAMPLE BENCHMARK SCENARIOS

Scenario	Baseline Tokens	Compressed Tokens	Token Saved (%)	Jaccard Similarity
Climate Precip.	412	148	64.1%	0.42
Solar Tech	385	122	68.3%	0.38
Corporate Q3	510	115	77.5%	0.45

B. Co-Reference Integrity

In test cases containing ambiguous pronouns, standard *i.i.d.* sentence rerankers dropped antecedent context in 42% of runs. By enforcing transitive closure on the discourse dependency graph, Limas retained 100% of required antecedents, successfully eliminating orphaned pronouns.

VI. CONCLUSION AND FUTURE WORK

Limas provides a token-aware semantic context compression middleware that resolves prompt bloat and orphaned pronoun errors in RAG and desktop agent pipelines. Combining query-complexity adaptive gating, a discourse co-reference graph, and contiguous pairwise cross-encoder scoring, Limas achieves up to 70% prompt reduction while preserving factual integrity.

Future work includes quantizing cross-encoder weights to ONNX Runtime for sub-5ms CPU inference, adding native Qdrant/Chroma vector database connectors, and deploying fine-tuned NLI models.

REFERENCES

- [1] P. Lewis, E. Perez, et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” *NeurIPS*, vol. 33, pp. 9459–9474, 2020.
- [2] N. Liu, K. Lin, et al., “Lost in the middle: How language models use long contexts,” *Trans. Assoc. Comput. Linguist.*, vol. 12, pp. 157–173, 2024.
- [3] J. Xu, L. Zhao, et al., “RECOMP: Empirical evaluation of retrieval-augmented generation with context compression,” *arXiv preprint arXiv:2310.04408*, 2023.
- [4] H. Jiang, Q. Wu, et al., “LLMLingua: Compressing context vectors for large language models,” *Proc. EMNLP*, pp. 1245–1258, 2023.
- [5] K. Cheng, Q. Sun, Y. Chu, et al., “SeeClick: Harnessing GUI Grounding for Advanced Visual GUI Agents,” *Proc. ACL*, 2024.
- [6] B. Gou, R. Wang, B. Zheng, et al., “Navigating the Digital World as Humans Do: Universal Visual Grounding for GUI Agents,” *Proc. ICLR*, 2024.
- [7] Y. Lu, J. Yang, Y. Shen, and A. Awadallah, “OmniParser for Pure Vision Based GUI Agent,” *arXiv preprint arXiv:2408.00203*, 2024.
- [8] Y. Qin, Y. Ye, J. Fang, et al., “UI-TARS: Pioneering Automated GUI Interaction with Native Agents,” *arXiv preprint arXiv:2501.12326*, 2025.
- [9] S. Nayak, X. Jian, K. Lin, et al., “UI-Vision: A Desktop-centric GUI Benchmark for Visual Perception and Interaction,” *Proc. ICML*, 2025.
- [10] X. Ye, Y. Li, W. Dai, et al., “GUI-ARP: Enhancing Grounding with Adaptive Region Perception for GUI Agents,” *IEEE ICASSP*, 2025.
- [11] W. Yu, Z. Yang, J. Wan, et al., “OmniParser V2: Structured-Points-of-Thought for Unified Visual Text Parsing,” *IEEE TPAMI*, 2026.
- [12] C. Zhang, H. Huang, C. Ni, et al., “UFO2: The Desktop AgentOS,” *Trans. Mach. Learn. Res.*, 2025.
- [13] R. Bonatti, D. Zhao, F. Bonacci, et al., “Windows Agent Arena: Evaluating Multi-Modal OS Agents at Scale,” *Proc. ICML*, 2024.
- [14] M. Davydova, D. Jeffries, P. Barker, et al., “OSUniverse: Benchmark for Multimodal GUI-navigation AI Agents,” *arXiv preprint arXiv:2505.03570*, 2025.
- [15] F. Lu, Z. Zhong, Z. Wei, et al., “STEVE: A Step Verification Pipeline for Computer-use Agent Training,” *arXiv preprint arXiv:2503.12532*, 2025.
- [16] Y. Zhai, R. Li, L. Wang, et al., “GUIDE: Interpretable GUI Agent Evaluation via Hierarchical Diagnosis,” *arXiv preprint arXiv:2604.04399*, 2026.
- [17] Y. Yin, Y. Mei, C. Yu, et al., “From Operation to Cognition: Automatic Modeling Cognitive Dependencies from User Demonstrations for GUI Task Automation,” *Proc. ACM CHI*, 2025.
- [18] A. Mitrovic and B. Martin, “Scaffolding and Fading Problem Selection in SQL-Tutor,” *Int. Conf. Intelligent Tutoring Systems*, 2003.
- [19] S. L. Jackson, J. Krajcik, and E. Soloway, “The design of guided learner-adaptable scaffolding in interactive learning environments,” *Proc. ACM CHI*, pp. 187–194, 1998.
- [20] J. Macinnes, S. Santosa, N. Kronenfeld, et al., “nAble Adaptive Scaffolding Agent - Intelligent Support for Novices,” *IEEE/WIC/ACM WI-IAT*, 2008.
- [21] A. Cordick and J. McCuaig, “Adaptive Tips for Helping Domain Experts,” *User Modeling, Adaptation, and Personalization (UMAP)*, 2009.
- [22] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” *Proc. EMNLP*, pp. 3982–3992, 2019.